

Data Cleaning

```
data <- read.csv("Dataset.csv", stringsAsFactors = FALSE)

# Convert all numeric columns (coerce blanks to NA)
data$Demand <- as.numeric(data$Demand)
data$Price <- as.numeric(data$Price)
data$Inflation <- as.numeric(data$Inflation)
data$Rate <- as.numeric(data$Rate)
data$USD <- as.numeric(data$USD)

# Check for missing values
cat("Missing values per column:\n")|
print(colSums(is.na(data)))

# Interpolate missing values using linear interpolation (zoo package)
data$Price <- na.approx(data$Price, na.rm = FALSE)
data$Inflation <- na.approx(data$Inflation, na.rm = FALSE)
data$Rate <- na.approx(data$Rate, na.rm = FALSE)
data$USD <- na.approx(data$USD, na.rm = FALSE)

# Confirm no remaining NAs
cat("\nMissing values after interpolation:\n")
print(colSums(is.na(data)))

# Remaining 1 missing value of Price
which(is.na(data$Price))
data$Price <- na.locf(data$Price, na.rm = FALSE)

# Confirm no remaining NAs
cat("\nMissing values after interpolation:\n")
print(colSums(is.na(data)))

# Total observations
cat("\nTotal observations:", nrow(data), "\n")
# Data runs from Q1 2010 to Q3 2025 = 63 quarters

# Convert to time series objects (quarterly, start Q1 2010)
GoldDemand <- ts(data$Demand, start = c(2010, 1), frequency = 4)
Price <- ts(data$Price, start = c(2010, 1), frequency = 4)
Inflation <- ts(data$Inflation, start = c(2010, 1), frequency = 4)
Rate <- ts(data$Rate, start = c(2010, 1), frequency = 4)
USD <- ts(data$USD, start = c(2010, 1), frequency = 4)
```

The dataset is loaded into R using `read.csv()`, with numeric columns explicitly coerced to numeric using `as.numeric()`. This converts any blank or non-numeric entries to NA. A missing value check using `colSums(is.na())` across Price, Inflation, Rate, and USD. These were imputed using linear interpolation via `na.approx()`, which estimates missing values by drawing a straight line between the nearest available observations, suitable for quarterly economic data where values change gradually. The remaining missing Price value was located using `which(is.na())` and filled with the previous quarter's value using `na.locf()`. Then, each variable was converted into a quarterly time series object using `ts()`, with the start period set to Q1 2010 and frequency set to 4.

```

Missing values per column:
  Time    Demand    Price Inflation    Rate    USD
    0         0         3         2         3         1

Missing values after interpolation:
  Time    Demand    Price Inflation    Rate    USD
    0         0         1         0         0         0
[1] 63

Missing values after interpolation:
  Time    Demand    Price Inflation    Rate    USD
    0         0         0         0         0         0

Total observations: 63

```

Output 1. *Missing value counts before and after imputation*

The dataset contains 63 quarterly observations with missing values detected across Price, Inflation, Rate, and USD, which were imputed to 0 using linear interpolation and last observation carried forward for the trailing NA in Price.

A. Regression with ARIMA Errors

1.

```

> xreg_full <- cbind(Price, Inflation, Rate, USD)
>
> model_A <- auto.arima(GoldDemand,
+                       xreg      = xreg_full,
+                       stepwise  = FALSE,
+                       approximation = FALSE,
+                       trace     = TRUE)

```

```
Best model: Regression with ARIMA(1,0,0)(0,0,2)[4] errors
```

Output 2. *auto.arima() model selection result*

The auto.arima() selected a regression model with ARIMA(1,0,0)(0,0,2)[4] errors

```

> summary(model_A)
Series: GoldDemand
Regression with ARIMA(1,0,0)(0,0,2)[4] errors

Coefficients:
      ar1      sma1      sma2  intercept    Price  Inflation    Rate    USD
    0.3327 -0.2286 -0.4682  2146.2301   0.3959   -16.0862   76.5183   1.3373
s.e.    0.1340   0.1388   0.1514   381.3442   0.1128    7.1205   31.4465   4.0086

sigma^2 = 17584:  log likelihood = -394.47
AIC=806.95  AICc=810.34  BIC=826.24

Training set error measures:
      ME      RMSE      MAE      MPE      MAPE      MASE      ACF1
Training set -3.07147 123.9013 90.46467 -1.274309 8.045296 0.6132564 -0.00561011

```

Output 3. *ARIMAX model summary - coefficients and training set error measures*

The chosen model is a Regression with ARIMA(1,0,0)(0,0,2)[4] errors. After adding the four macroeconomic variables to the regression, the remaining error structure follows a first-order autoregressive process with two seasonal moving average terms for each quarter.

The RMSE value of 123.90 tonnes and MAE of 90.46 tonnes show that the forecasted gold demand differs from the actual demand by about 90 to 124 tonnes on average each quarter, which is modest considering that quarterly demand ranges between approximately 778 and 2,084 tonnes in this sample. The MAPE of 8.05% further confirms an acceptable level of forecast accuracy.

The residual diagnostics of the model fit the data well. The ACF1 value of -0.006 is very close to zero; there is almost no autocorrelation left in the residuals, suggesting that the ARIMA error structure captures the remaining time-series patterns after the regression variables.

```
# Coefficient table with standard errors and t-stats
coef_table <- coef(model_A)
se_table   <- sqrt(diag(vcov(model_A)))
t_stats    <- coef_table / se_table
p_values    <- 2 * pnorm(abs(t_stats), lower.tail = FALSE)

cat("\n--- Coefficient Table ---\n")
coef_df <- cbind(
  Estimate = coef_table,
  Std.Error = se_table,
  t.value = t_stats,
  p.value = p_values
)

printCoefmat(coef_df, digits = 4, P.values = TRUE, has.Pvalue = TRUE)

cat("\nAIC:", AIC(model_A), "\n")
cat("\nBIC:", BIC(model_A), "\n")
```

The function `coef(model_A)` extracts the estimated coefficients from the ARIMAX model, while `sqrt(diag(vcov(model_A)))` calculates their standard errors from the variance-covariance matrix. The t-statistics are then computed by dividing each coefficient by its standard error, showing how far each estimate is from zero in standard error units. P-values are obtained using a two-tailed normal distribution (`2 * pnorm(abs(t_stats), lower.tail = FALSE)`), where values below 0.05 indicate statistical significance. Finally, the estimates, standard errors, t-values, and p-values are combined using `cbind()` and displayed neatly with `printCoefmat()`, which also adds significance stars based on p-value levels.

--- Coefficient Table ---					
	Estimate	Std.Error	t.value	p.value	
ar1	0.3327	0.1340	2.483	0.013012	*
sma1	-0.2286	0.1388	-1.647	0.099554	.
sma2	-0.4682	0.1514	-3.092	0.001987	**
intercept	2146.2301	381.3442	5.628	1.82e-08	***
Price	0.3959	0.1128	3.510	0.000448	***
Inflation	-16.0862	7.1205	-2.259	0.023875	*
Rate	76.5183	31.4465	2.433	0.014962	*
USD	1.3373	4.0086	0.334	0.738677	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1					
AIC: 806.9469					
BIC: 826.2351					

Output 4. Regression with ARIMA Errors Model Coefficients and Hypothesis Testing Results

Table 1. Regression with ARIMA Errors Model Coefficients and Hypothesis Testing Results

Variable	Estimate	Std. Error	t-value	p-value	Interpretation
ar1	0.3327	0.1340	2.4834	0.0130	Significant AR component in residuals
sma1	-0.2286	0.1388	-1.6470	0.0996	Not significant at 5% level
sma2	-0.4682	0.1514	-3.0921	0.0020	Significant seasonal MA component
intercept	2146.2301	381.3442	5.6281	0.0000	Significant baseline gold demand when predictors are 0
Price	0.3959	0.1128	3.5101	0.0004	Significant positive relationship with gold demand
Inflation	-16.0862	7.1205	-2.2591	0.0239	Significant negative relationship with gold demand
Rate	76.5183	31.4465	2.4333	0.0150	Significant positive relationship with gold demand
USD	1.3373	4.0086	0.3336	0.7387	Not statistically significant

Price (Gold Price)

When the gold price increases by 1 dollar per ounce, global gold demand increases by about 0.40 tonnes, holding other variables constant. The positive sign is expected because during periods of rising prices, investors may expect prices to continue increasing and purchase more gold for investment purposes. This reflects strong investor sentiment and speculative demand in the gold market.

Inflation

A one-unit increase in inflation is linked to a decrease of about 16.09 tonnes in gold demand. Although gold is often considered an inflation hedge, according to the IMF, global inflation rose to 8.8% in 2022 and remained elevated at 6.6% in 2023, which reduced consumers' purchasing power and prompted central banks to raise interest rates. (International Monetary Fund, 2023) As gold does not generate interest income, some investors shifted to interest-bearing assets, contributing to lower gold demand.

Rate (Interest Rate)

A 1% increase in interest rates is associated with an increase of approximately 76.52 tonnes in gold demand. Normally, higher interest rates reduce gold demand because gold does not provide interest income. However, in this dataset, higher rates may reflect periods of economic uncertainty where investors shifted towards gold as a safe-haven asset where strong central bank buying was powerful enough to offset that effect, so gold demand still increased overall. (World Gold Council, 2024)

USD (USD Index)

Since USD is not significant ($p = 0.739$), it does not contribute meaningfully to explaining gold demand in this model and could potentially be removed without affecting model fit. The US dollar does not add additional explanatory power on gold demand after accounting for gold price, inflation, and interest rates. One possible reason is that the effect of the USD is already indirectly captured through the gold price variable.

ARIMA Error Terms

The AR(1) coefficient of 0.3327 suggests there is some positive autocorrelation; shocks from one quarter can still affect the next quarter. The seasonal MA(2) coefficient of -0.4682 shows that seasonal effects tend to reverse after two quarters. The SMA(1) term is slightly insignificant but was still retained by `auto.arima()` because it helps improve the overall model fit and lowers the AIC value.

Summary

Based on the coefficient estimates, **Rate (Interest Rate)** had the strongest coefficient at 76.5183; it has the largest absolute effect on Gold Demand per unit change among the four predictors. 3 out of 4 predictors are statistically significant at the 5% level, with only USD failing to reach significance, suggesting the model as a whole captures the key macroeconomic drivers of gold demand well. The intercept carries a t-value of 5.628 ($p < 0.001$), Gold Price ($p = 0.0004$), Inflation ($p = 0.0239$), and Rate ($p = 0.0150$) are all statistically significant at the 5% level. The AIC of **806.95** and AICc of 810.34 confirm good model fit relative to the OLS baseline of 820.39.

2.

```
residuals_A <- residuals(model_A)

# PACF Plot
Pacf(residuals_A, main = "PACF of Residuals (Model A)")

# Normality of residuals
shapiro_test <- shapiro.test(residuals_A)
cat("\nShapiro-Wilk Normality Test:\n")
print(shapiro_test)

# Full residual diagnostic plot (checkresiduals)
checkresiduals(model_A)
```

Residuals are extracted from the fitted model, and then a PACF plot is used to visually examine the partial autocorrelation of the residuals at different lags. In addition, a Shapiro-Wilk test was conducted to test for normality, while `checkresiduals()` provided a combined diagnostic output including the residual time plot, ACF plot, histogram, and Ljung-Box test.

```
Shapiro-Wilk Normality Test:

      Shapiro-Wilk normality test

data:  residuals_A
W = 0.96514, p-value = 0.07131

      Ljung-Box test

data:  Residuals from Regression with ARIMA(1,0,0)(0,0,2)[4] errors
Q* = 3.6693, df = 5, p-value = 0.5979

Model df: 3.    Total lags used: 8
```

Output 5. *Shapiro-Wilk normality test and Ljung-Box test on ARIMAX residuals*

Shapiro-Wilk Normality Test ($W = 0.965$, $p = 0.071$)

The null hypothesis of normality cannot be rejected at the 5% significance level, confirming the residuals are approximately normally distributed. Therefore, the model assumptions are reasonably satisfied, and the coefficient estimates are reliable.

Ljung-Box Test ($Q = 3.669$, $df = 5$, $p = 0.5979$)

The `checkresiduals()` function applies this test while adjusting the degrees of freedom to account for the estimated ARIMA parameters (model $df = 3$, total lags = 8). The high p-value indicates that we fail to reject the null hypothesis of no autocorrelation, confirming that the residuals behave like white noise.

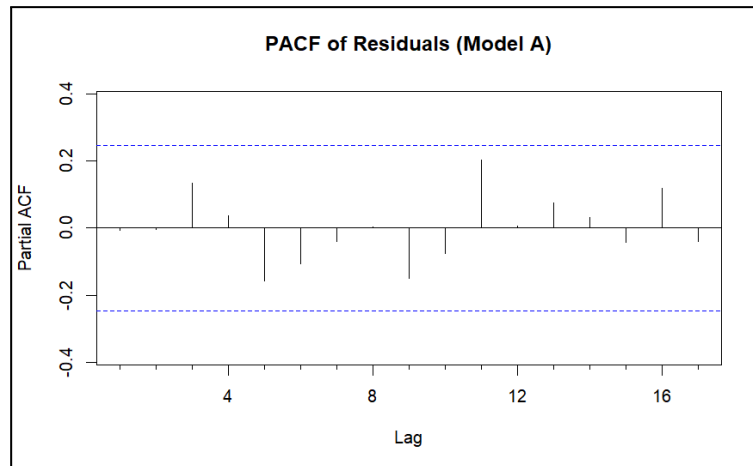


Figure 1. *PACF of residuals from Regression with ARIMA Errors*

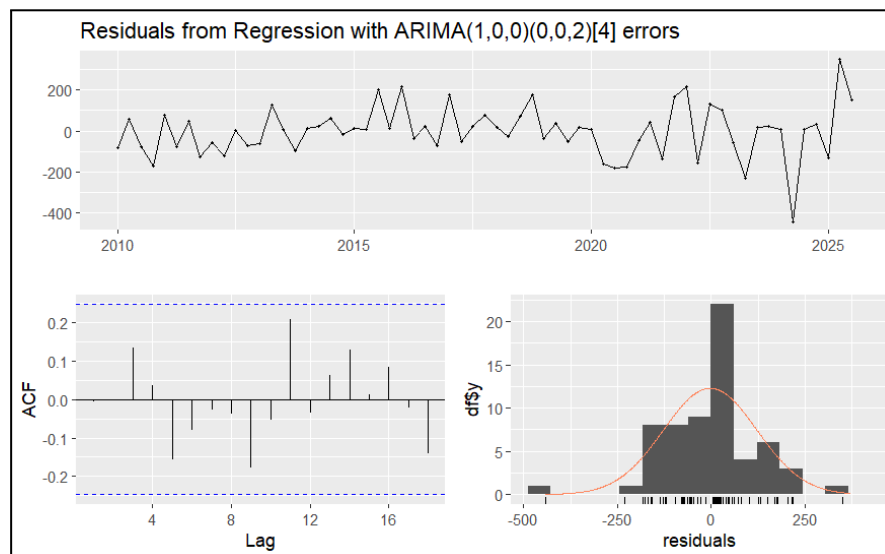


Figure 2. *Residual diagnostics - time plot, ACF and histogram from checkresiduals()*

Residual Time Plot: The residuals mostly fluctuate around zero with no clear trend or lie between approximately -200 and +200 from Q1 2010 to Q3 2025, which suggests the errors behave like white noise. However, there is one noticeable negative spike of about -450 tonnes around 2024. A large positive residual appears near 2025 around +350, highlighting the model's limitation in handling structural outliers.

Residual ACF: All autocorrelation values across the different lags fall within the 95% confidence bounds. A positive autocorrelation is observed at lag 3 (about 0.13), while negative autocorrelations are observed at lags 5 and 6 (about -0.15 and -0.10, respectively). Lag 11 shows the largest positive autocorrelation (about 0.20), but all remain statistically insignificant. This shows there is no remaining meaningful autocorrelation in the residuals.

PACF: All partial autocorrelation spikes similarly remain within the confidence bounds, with no lag showing a meaningful spike, further supporting that there is no leftover linear relationship over time, and confirming that the residuals are close to white noise.

Summary

The residuals satisfying white noise conditions imply that the ARIMA(1,0,0)(0,0,2)[4] error structure has successfully captured all systematic autocorrelation left after the regression, meaning the model is well-specified, and the coefficient estimates and forecast intervals are statistically reliable.

3.

```
# Fit a plain OLS (ARIMA(0,0,0) errors) to compare
model_OLS <- Arima(GoldDemand,
                  order = c(0, 0, 0),
                  xreg = xreg_full)
cat("OLS (no ARIMA errors) AIC:", AIC(model_OLS), "\n")

# Check OLS residuals for autocorrelation
lb_OLS <- Box.test(residuals(model_OLS), lag = 10, type = "Ljung-Box")
cat("Ljung-Box on OLS residuals:\n")
print(lb_OLS)

cat("\nARIMA-X Model AIC:", AIC(model_A), "\n")
cat("Full ARIMAX Model specification:\n")
print(model_A)
```

A plain OLS regression (ARIMA(0,0,0) errors) is fitted using Arima() with the same four predictors as a baseline comparison. A Ljung-Box test is then applied to the OLS residuals to check for autocorrelation, and the AIC of the OLS model was compared with the AIC of the ARIMAX model to evaluate whether adding the ARIMA error structure improves model performance.

```

===== A.3: ARIMA ERROR STRUCTURE =====
OLS (no ARIMA errors) AIC: 820.3875
Ljung-Box on OLS residuals:

      Box-Ljung test

data: residuals(model_OLS)
X-squared = 26.741, df = 10, p-value = 0.002861

ARIMA-X Model AIC: 806.9469
Full ARIMAX Model specification:
Series: GoldDemand
Regression with ARIMA(1,0,0)(0,0,2)[4] errors

Coefficients:
      ar1      sma1      sma2  intercept  Price  Inflation      Rate      USD
0.3327 -0.2286 -0.4682  2146.2301  0.3959  -16.0862  76.5183  1.3373
s.e.  0.1340  0.1388  0.1514   381.3442  0.1128    7.1205  31.4465  4.0086

sigma^2 = 17584: log likelihood = -394.47
AIC=806.95  AICc=810.34  BIC=826.24

```

Output 6. OLS baseline model AIC and Ljung-Box test on OLS residuals

The OLS baseline model (ARIMA(0,0,0) errors) was first fitted using the same four predictors to check whether an ARIMA error structure was needed. A Ljung–Box test was then applied to the OLS residuals, and AIC values were compared between models to determine if adding ARIMA errors improved the fit.

The Ljung–Box test on the OLS residuals produced $X^2 = 26.741$ (df = 10, p = 0.0029), indicating strong evidence of autocorrelation and rejecting the null hypothesis of no serial correlation at the 1% significance level. In addition, the OLS model AIC was 820.39, compared to 806.95 for the ARIMAX model, representing a reduction of 13.44 AIC units in favour of the ARIMAX specification. The final model is a regression with ARIMA(1,0,0)(0,0,2)[4] errors, with coefficients reported A.1 section.

The significant Ljung–Box result shows that the OLS residuals have clear serial dependence, which OLS alone cannot capture. This violates the white noise assumption and means the coefficient estimates are less reliable for inference. The ARIMA(1,0,0)(0,0,2)[4] error structure addresses this issue by modelling the autocorrelation directly.

The AR(1) term (0.3327) shows that the current residual is influenced by the residual from the previous quarter, while the seasonal MA(2) term captures yearly dependence at lags 4 and 8 (–0.2286 and –0.4682), while the seasonal MA(1) term (–0.2286), although only marginally significant (p = 0.0996), was kept by `auto.arma()` because it helps reduce AICc and improves overall model fit.

Summary

Overall, the ARIMA error structure is necessary. The 13.44 AIC unit improvement over OLS, combined with the Ljung-Box confirmation of white noise residuals in A.2 ($p = 0.5979$), validates that the $ARIMA(1,0,0)(0,0,2)[4]$ captures the remaining temporal dependence not explained by the macroeconomic predictors, making it a necessary improvement over the OLS model.

4.

```
last_row <- tail(data, 1)
xreg_future <- matrix(
  rep(c(last_row$Price, last_row$Inflation, last_row$Rate, last_row$USD), 4),
  nrow = 4, byrow = TRUE,
  dimnames = list(NULL, c("Price", "Inflation", "Rate", "USD"))
)

forecast_A <- forecast(model_A, xreg = xreg_future, h = 4)
print(forecast_A)

autoplot(forecast_A) +
  ggtitle("4-Quarter Ahead Forecast: Gold Demand (ARIMAX Model)") +
  xlab("Year") + ylab("Gold Demand (Tonnes)") +
  theme_minimal()
```

The last row of predictor values (Price, Inflation, Rate, USD) is taken using `tail()` and repeated four times to create a future regressor matrix, representing a naive assumption that predictors remain constant over the forecast horizon. This matrix is then used in `forecast()` with the fitted ARIMAX model to generate a 4-quarter-ahead forecast, which is then plotted using `autoplot()`.

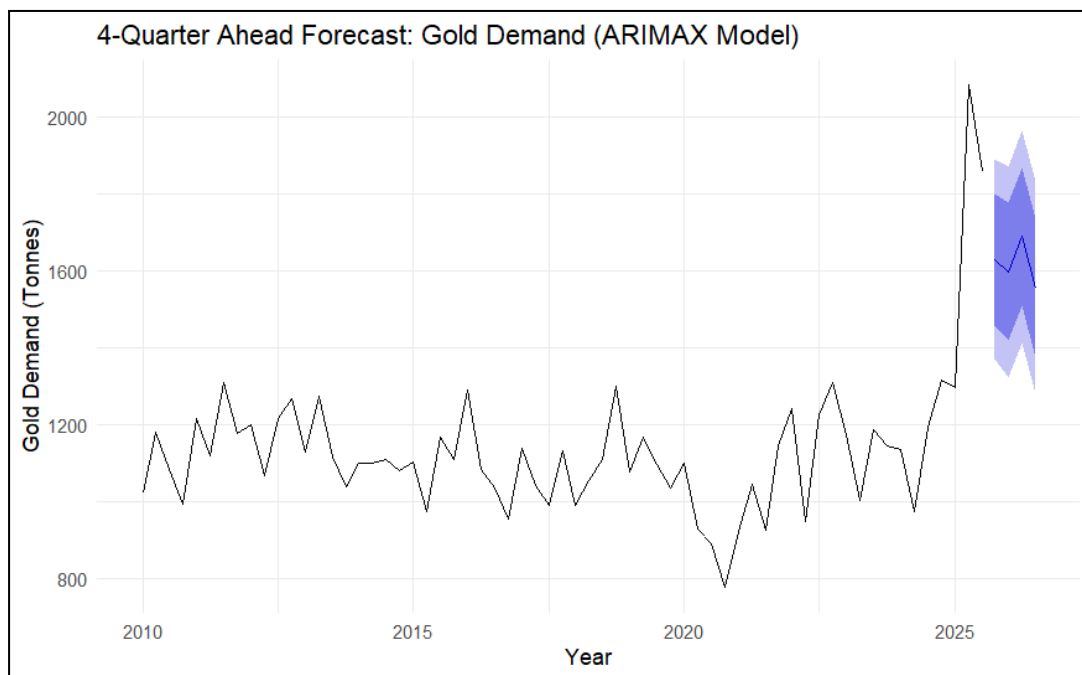


Figure 3. 4-quarter ahead forecast of gold demand using ARIMAX model

Table 2. 4-quarter ahead point forecasts and 80%/95% prediction intervals (ARIMAX model)

Period	Point Forecast	Lower 80%	Upper 80%	Lower 95%	Upper 95%
2025 Q4	1630.266	1460.301	1800.230	1370.327	1890.204
2026 Q1	1598.745	1419.629	1777.861	1324.810	1872.679
2026 Q2	1689.216	1509.115	1869.317	1413.775	1964.657
2026 Q3	1555.742	1375.532	1735.952	1280.135	1831.349

The 4-quarter-ahead forecast projects gold demand declining from approximately 1,600 tonnes in Q4 2025 towards roughly 1,400-1,500 tonnes by Q3 2026, with widening prediction intervals. More specifically, the forecasts range between 1,556 and 1,689 tonnes, peaking in Q2 2026 (1,689 tonnes) and reaching the lowest point in Q3 2026 (1,556 tonnes).

This pattern is economically reasonable because the underlying ARIMA(1,0,0)(0,0,2)[4] structure is mean-reverting. It adjusted downward after the extreme spike of 2,083.8 tonnes in Q2 2025, which is far above the historical range of roughly 778-1,316 tonnes observed between 2010 and 2024. In this sense, the model treats the 2025 surge as a temporary shock rather than a permanent structural shift.

However, the forecasted levels remain relatively high compared to the long-run average of about 1,100 tonnes. This is mainly due to the assumption that the most recent predictor values such as the elevated gold price of 3,280 USD/oz and interest rate of 4.29% remain constant throughout the forecast horizon, which keeps demand projections upward.

Overall, while the direction and behaviour of the forecast are reasonable under the model assumptions, the constant-predictor assumption means the point forecasts should still be interpreted with caution.

B. Advanced Forecasting Methods

B.a Forecast Combination (Ensemble)

a.1.

```
# Train/test split: hold out last 8 quarters as test set
h_test      <- 8
n_total     <- length(GoldDemand)
train_end   <- time(GoldDemand)[n_total - h_test]

GoldDemand_train <- window(GoldDemand, end = train_end)
GoldDemand_test  <- window(GoldDemand, start = time(GoldDemand)[n_total - h_test + 1])

# — B.a Forecast Combination (Ensemble) —

# 1. Seasonal Naive
snaive_fc <- snaive(GoldDemand_train, h = h_test)

# 2. Auto ETS
ets_model <- ets(GoldDemand_train)
ets_fc    <- forecast(ets_model, h = h_test)

# 3. Auto ARIMA (univariate)
arma_uni  <- auto.arima(GoldDemand_train,
                        stepwise = FALSE, approximation = FALSE)
arma_fc   <- forecast(arma_uni, h = h_test)

# 4. STL decomposition forecast
stl_fc    <- stlf(GoldDemand_train, h = h_test)

# Ensemble = simple average
ensemble_mean <- (snaive_fc$mean + ets_fc$mean +
                  arma_fc$mean + stl_fc$mean) / 4
```

Train/Test Split: `window()` splits the series into training (all but last 8 quarters) and test sets, using `time()` to locate the exact split

Seasonal Naïve: `snaive()` forecasts each quarter as equal to the same quarter one year ago, serving as a simple seasonal benchmark

Auto ETS: `ets()` selects the best exponential smoothing model by minimising AICc; `forecast()` next 8-quarter ahead predictions

Auto ARIMA: `auto.arima()` exhaustively searches ARIMA orders via AICc with exact likelihood (`stepwise=FALSE`, `approximation=FALSE`), then forecasts 8-quarter ahead

STL Forecast: `stlf()` decomposes the series using Loess, models the seasonally adjusted component with ETS, then reattaches the seasonal component for final forecasts.

Ensemble Average: The four mean forecasts are summed and divided by 4, combining each model's strengths to reduce overall forecast error

```
# Accuracy - individual models
acc_snaive <- accuracy(snaive_fc, GoldDemand_test)
acc_ets <- accuracy(ets_fc, GoldDemand_test)
acc_arima <- accuracy(arima_fc, GoldDemand_test)
acc_stl <- accuracy(stl_fc, GoldDemand_test)

# Accuracy - ensemble (manual)
ens_errors <- GoldDemand_test - ensemble_mean
rmse_ens <- sqrt(mean(ens_errors^2))
mae_ens <- mean(abs(ens_errors))
mape_ens <- mean(abs(ens_errors / GoldDemand_test)) * 100

cat("\n--- Ensemble Accuracy (Test Set) ---\n")
cat("RMSE:", round(rmse_ens, 3), "\n")
cat("MAE: ", round(mae_ens, 3), "\n")
cat("MAPE:", round(mape_ens, 3), "%\n")

# Comparison table
acc_table <- rbind(
  SNaive = acc_snaive[2, c("RMSE", "MAE", "MAPE")],
  ETS = acc_ets[2, c("RMSE", "MAE", "MAPE")],
  ARIMA = acc_arima[2, c("RMSE", "MAE", "MAPE")],
  STL = acc_stl[2, c("RMSE", "MAE", "MAPE")],
  Ensemble = c(RMSE = rmse_ens, MAE = mae_ens, MAPE = mape_ens)
)
print(round(acc_table, 3))
```

Individual Accuracy: accuracy(forecast, test) returns train and test error metrics; The second row [2,] extracts the test-set row containing RMSE, MAE, and MAPE.

Ensemble Accuracy: Since accuracy() requires a forecast object, ensemble errors are computed manually by subtracting the ensemble forecast values (ensemble_mean) from the test set and applying the RMSE, MAE, and MAPE formulas directly.

Comparison Table: rbind() stacks all five models' test-set metrics into one table for comparison.

```
# Plot
autoplot(GoldDemand) +
  autolayer(snaive_fc$mean, series = "SNaive") +
  autolayer(ets_fc$mean, series = "ETS") +
  autolayer(arima_fc$mean, series = "ARIMA") +
  autolayer(stl_fc$mean, series = "STL") +
  autolayer(ensemble_mean, series = "Ensemble") +
  ggtitle("Individual vs Ensemble Forecasts of Gold Demand") +
  xlab("Year") + ylab("Demand (tonnes)") +
  guides(colour = guide_legend(title = "Model"))
```

The autoplot() plots the full GoldDemand series as the base layer. Each autolayer() was then used to add the forecasts from each model to the same graph, with the series argument providing labels for the legend. The ggtitle(), xlab(), and ylab() functions were used to add a title and axis labels, while guides() was used to label the colour legend as "Model" .

--- Ensemble Accuracy (Test Set) ---				
RMSE: 446.418				
MAE: 286.413				
MAPE: 16.83 %				
	RMSE	MAE	MAPE	
SNaive	455.106	264.388	14.864	
ETS	433.925	279.977	16.581	
ARIMA	484.169	378.073	24.353	
STL	437.911	275.953	16.135	
Ensemble	446.418	286.413	16.830	
	Point Forecast	Lo 100	Hi 100	
2023 Q4	1131.960	1085.719	1189.719	
2024 Q1	1134.560	1085.719	1212.374	
2024 Q2	1128.965	1085.719	1189.719	
2024 Q3	1129.733	1085.719	1189.719	
2024 Q4	1131.960	1085.719	1189.719	
2025 Q1	1134.560	1085.719	1212.374	
2025 Q2	1128.965	1085.719	1189.719	
2025 Q3	1129.733	1085.719	1189.719	

Output 7. Ensemble model accuracy - test set RMSE, MAE and MAPE

The ensemble model did not perform better than all of the individual forecasting models. Based on the accuracy results, the ETS model achieved the lowest RMSE (433.925), while the STL model produced the lowest MAE (275.953) and MAPE (16.135%). These results were better than the ensemble model, which recorded an RMSE of 446.418, MAE of 286.413, and MAPE of 16.830%. However, the ensemble model still outperformed the Seasonal Naïve model and the ARIMA model, which had higher forecast errors.

One possible reason for this result is the sharp increase in gold demand during Q2 2025, where demand rose to approximately 2,083 tonnes. This unusual spike was not captured well by any of the forecasting models. Since all models made similar forecasting errors, averaging their forecasts did not improve performance. The main advantage of an ensemble forecast is that errors from different models can offset each other, but this benefit is reduced when all models miss the same pattern.

In addition, the ARIMA model performed noticeably worse than the other models, with the highest MAPE of 24.353%. Because the ensemble forecast was calculated using an equal-weighted average, the poorer ARIMA forecasts dragged the overall MAPE ensemble forecast to 16.83% which was worse than both ETS (16.581%) and STL (16.135%) forecasts. As a result, the ensemble model was unable to outperform the best individual models.

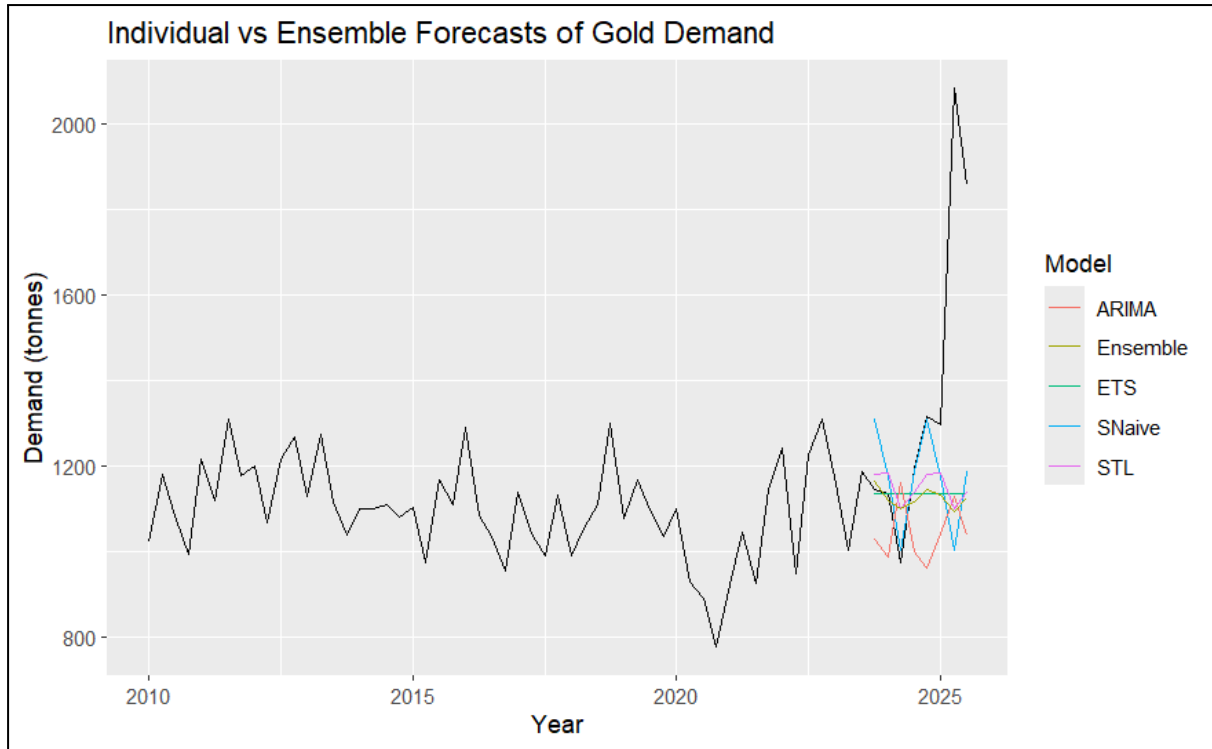


Figure 4. *Individual vs Ensemble forecasts of gold demand versus actual values*

The forecast plot shows the Gold Demand series from 2010 to mid-2025, with forecasts for the 8-quarter test period forecasts overlaid from Q4 2023 onwards. A notable feature of the plot is the sharp increase in gold demand in Q2 2025, which reached approximately 2,083 tonnes. This value is double the historical average than the historical average and was not predicted by any of the forecasting models.

All five forecasts remain tightly clustered between 950 to 1,250 tonnes during the test period. This suggests that the models mainly relied on historical patterns and were unable to anticipate the sudden surge in demand. The Seasonal Naïve model produced the most fluctuating forecasts because it repeats the seasonal pattern from the previous year. In contrast, the ARIMA model generated the lowest forecasts, dipping close to 900 tonnes.

The ETS, STL, and Ensemble forecasts were very similar, generally remaining between 1,050 and 1,150 tonnes. As expected, the Ensemble forecast lay between the four individual model forecasts as expected from averaging. The fact that all forecasts were clustered closely together indicates that the models produced similar predictions. As a result, the ensemble forecast did not gain much benefit from combining different models, since there was limited diversity among the individual forecasts.

a.2

Benefits of Combining Forecasts in Volatile Markets Like Gold

1. Reduces Forecast Variability

Each individual model captures different aspects of the data. For example, ETS captures level and trend smoothing, ARIMA captures autocorrelation structure, STL captures seasonal decomposition, and Seasonal Naïve captures repeating quarterly cycles. When averaged, random errors across models partially cancel out, reducing overall forecast variance.

2. Robustness to Model Uncertainty

Gold demand is influenced by many factors, including investment demand, jewellery consumption, central bank purchases, and geopolitical events. Since no single forecasting model can capture all of these influences perfectly, relying on one model may lead to inaccurate forecasts. Combining forecasts from several models reduces the risk of depending on a single model and provides a more balanced prediction. This is especially useful in the gold market, where demand patterns can change unexpectedly, such as the large increase in Q2 2025.

3. Reduced Sensitivity to Structural Breaks

Volatile commodity markets like gold are prone to sudden changes. For example, COVID-19 in 2020, Fed rate hikes in 2022, and increased central bank purchases. No single model anticipates these events well. An ensemble spreads this risk, for example, if ARIMA overreacts to a recent shock while ETS smooths over it, the average produces a more stable, less extreme forecast. In this dataset, ARIMA produced the worst individual MAPE of 24.353%, the ensemble moderated this by averaging in the more stable ETS and STL forecasts, achieving a lower MAPE of 16.830%.

Limitation

The benefit was limited here because all four models underestimated the 2025 spike. Since the model errors are highly correlated, averaging provides little improvement. This highlights that ensemble methods work best when individual models have different types of errors.

B.b Bootstrap & Bagging

b.1.

```
set.seed(123)
bagged_model <- baggedModel(
  GoldDemand_train,
  bootstrapped_series = bld.mbb.bootstrap(GoldDemand_train, 20)
)

bagged_fc <- forecast(bagged_model, h = h_test)
print(bagged_fc)

autoplot(bagged_fc) +
  ggtitle("Bagged Model Forecast of Gold Demand (B = 20)") +
  xlab("Year") + ylab("Demand (tonnes)")

acc_bagged <- accuracy(bagged_fc, GoldDemand_test)
cat("\n--- Bagged Model Accuracy (Test Set) ---\n")
print(round(acc_bagged, 3))
```

set.seed(123) ensures reproducibility of the bootstrap sampling

bld.mbb.bootstrap() used to create 20 bootstrapped versions of the training series using the Moving Block Bootstrap method, which resamples blocks of consecutive observations to preserve the autocorrelation structure

baggedModel() fits an ETS model to each of the 20 bootstrapped series,

forecast() averages all 20 resulting forecasts into a single bagged forecast

accuracy() then evaluates the bagged forecast against the test set using RMSE, MAE, and MAPE.

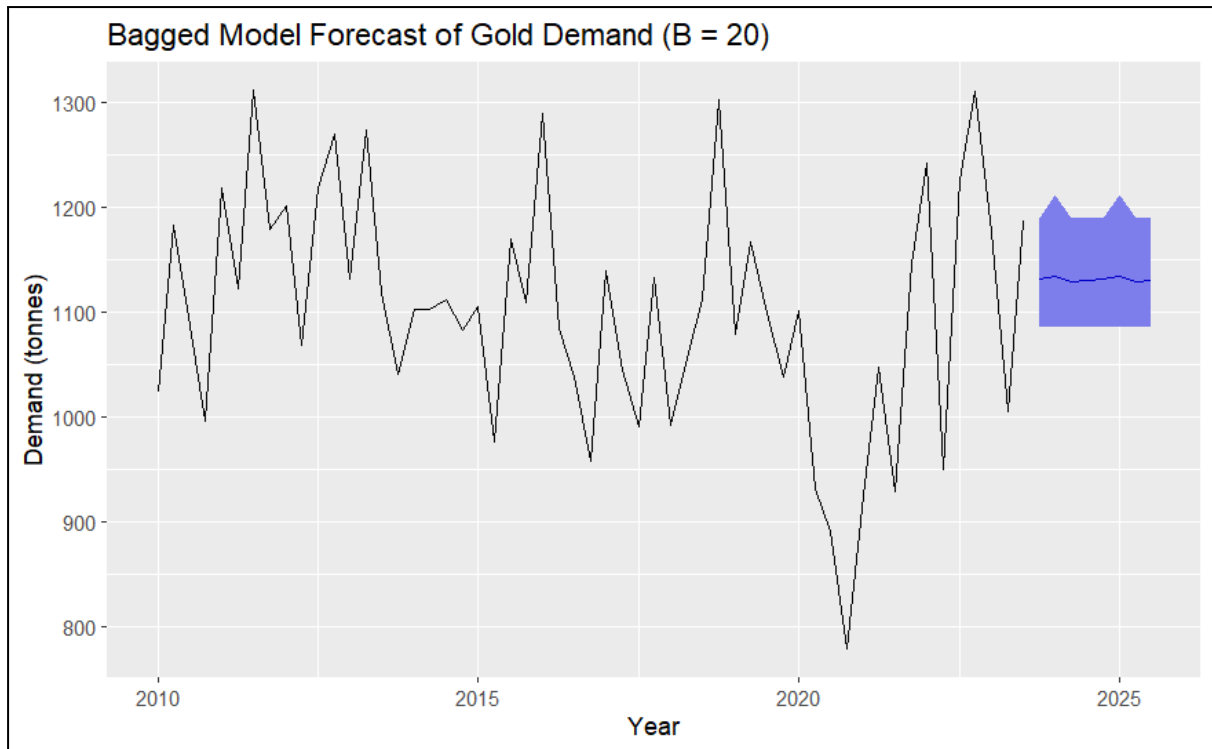


Figure 5. Bagged ETS model forecast of gold demand with 80% prediction interval (B=20)

The bagged model forecasts Gold Demand between approximately 1,100 and 1,200 tonnes across the 8-quarter test window, with the shaded prediction interval ranging from roughly 1,085 to 1,190 tonnes. The point forecasts show a mild seasonal undulation, reflecting the averaged seasonal patterns captured across the 20 bootstrapped series. However, similar to the ensemble, the bagged model completely missed the Q2 2025 spike to 2,083 tonnes, as the prediction interval's upper bound of approximately 1,190 tonnes falls far short of the actual value. The relatively narrow prediction interval suggests the 20 bootstrapped series produced similar forecasts, showing low variability across bootstrap replicates which is expected given Gold Demand's historically stable range of 950-1,300 tonnes prior to the anomalous 2025 surge.

b.2.

--- Bagged Model Accuracy (Test Set) ---								
	ME	RMSE	MAE	MPE	MAPE	MASE	ACF1	Theil's U
Training set	0.424	101.654	82.405	-0.859	7.609	0.660	0.094	NA
Test set	244.883	437.318	283.024	12.854	16.760	2.268	0.500	1.257

Output 7. Bagged ETS model accuracy - training and test error measures

Bagging did not significantly improve forecast accuracy in this study. The bagged model produced a test RMSE of 437.318, MAE of 283.024, and MAPE of 16.760%. Although these results were slightly better than the simple ensemble model (RMSE = 446.418, MAPE = 16.830%), they were still worse than the best-performing individual models, ETS (RMSE = 433.925) and STL (MAPE = 16.135%). In addition, the large difference between the training RMSE (101.654) and test RMSE (437.318) suggests that the bagged model fitted the training data well but did not generalise effectively to the test period. This may be due to the sharp increase in gold demand during 2025, which could not be predicted from the historical patterns used to generate the bootstrap samples.

Bagging is generally useful for time series with high volatility, structural instability, or irregular patterns, where a single model fitted to one realisation of the data may be overly sensitive to noise. By averaging across 20 bootstrapped series, bagging reduces variance in the forecast and produces more stable estimates. However, its effectiveness is limited when forecast errors are caused by structural breaks or unexpected events. In this case, the sudden surge in gold demand in 2025 represented a genuine change in the underlying pattern rather than random variation. Since all bootstrap samples were generated from the same historical data, they were unable to capture this unusual event, resulting in limited improvement in forecast performance.

C Model Evaluation and Selection

1. RMSE, MAPE, MAE

```
n_train <- n_total - h_test
xreg_train <- xreg_full[1:n_train, ]
xreg_test <- xreg_full[(n_train + 1):n_total, ]

set.seed(42)
model_A_train <- auto.arima(GoldDemand_train,
                           xreg      = xreg_train,
                           stepwise  = FALSE,
                           approximation = FALSE,
                           seasonal  = TRUE)
fc_A_test <- forecast(model_A_train, xreg = xreg_test, h = h_test)

calc_metrics <- function(errors, actual) {
  c(RMSE = sqrt(mean(errors^2)),
    MAE  = mean(abs(errors)),
    MAPE = mean(abs(errors / actual)) * 100)
}

final_acc <- rbind(
  `A: ARIMAX` = calc_metrics(GoldDemand_test - fc_A_test$mean, GoldDemand_test),
  `B-a: Ensemble` = calc_metrics(GoldDemand_test - ensemble_mean, GoldDemand_test),
  `B-b: Bagged` = calc_metrics(GoldDemand_test - bagged_fc$mean, GoldDemand_test)
)

cat("\n=== Section C: Test-Set Accuracy (Last 8 Quarters) ===\n")
print(round(final_acc, 3))
```

The training set was created using `n_train <- n_total - h_test`, which leaves the first 55 quarters for model estimation and reserves the last 8 quarters for testing, ensuring all models were trained on the same data for a fair comparison. The external regressors were split into training and testing sets using `xreg_train` and `xreg_test`.

The ARIMAX model was refitted using `auto.arima()` with `stepwise = FALSE` and `approximation = FALSE`, then used to forecast the 8-quarter test period with `xreg_test`. A custom `calc_metrics()` function computed RMSE, MAE, and MAPE consistently across all models, and the results were combined into a single comparison table using `rbind()`.

=== Section C: Test-Set Accuracy (Last 8 Quarters) ===			
	RMSE	MAE	MAPE
A: ARIMAX	538.078	428.530	27.768
B-a: Ensemble	446.418	286.413	16.830
B-b: Bagged	437.318	283.024	16.760

Output 8. Test set accuracy - ARIMAX, Ensemble, Bagged ETS models (last 8 quarters)

The test-set accuracy results show that the Bagged ETS model performed best across all three evaluation measures. It achieved the lowest RMSE (437.318), MAE (283.024), and MAPE (16.760%). The Ensemble model ranked second with an RMSE of 446.418 and a

MAPE of 16.830%, which were only slightly higher than the Bagged model. The ARIMAX model produced the weakest results, with an RMSE of 538.078 and a MAPE of 27.768%.

The weak ARIMAX performance may be due to the assumption that predictor values remained constant over the 8-quarter forecast period. Since gold prices rose significantly during 2024-2025, these fixed values likely did not reflect actual market conditions, increasing forecast errors. The model was also highly influenced by the exceptional demand spike in 2025 (2,083 tonnes), whereas the Ensemble and Bagged models relied more on historical patterns and were less affected by predictor inaccuracies.

```
# Plot: all three forecasts vs actuals
autoplot(GoldDemand_train) +
  autolayer(fc_A_test$mean, series = "A: ARIMAX", lwd = 1.2) +
  autolayer(ensemble_mean, series = "B-a: Ensemble", lwd = 1.2) +
  autolayer(bagged_fc$mean, series = "B-b: Bagged", lwd = 1.2) +
  autolayer(GoldDemand_test, series = "Actual", lwd = 1.2) +
  scale_colour_manual(values = c("A: ARIMAX" = "steelblue",
                                "B-a: Ensemble" = "darkgreen",
                                "B-b: Bagged" = "purple",
                                "Actual" = "black")) +
  ggtitle("Section C: Forecast Comparison vs Actual (Test Set)") +
  xlab("Year") + ylab("Gold Demand (Tonnes)") +
  guides(colour = guide_legend(title = "Model")) +
  theme_minimal()
```

autoplot() displays the training data, while autolayer() adds the ARIMAX, Ensemble, Bagged, and actual test-set series for comparison. scale_colour_manual() assigns custom colours, and the remaining functions add titles, labels, a legend, and a minimal theme to improve the plot's readability.

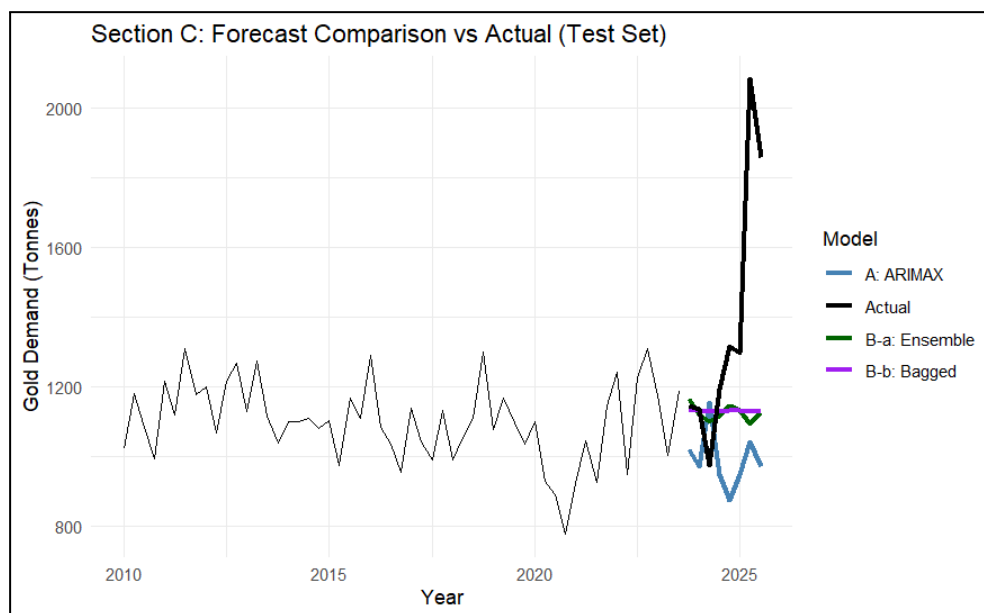


Figure 6. Forecast Comparison vs Actual Gold Demand (Test Set)

The plot displays the full Gold Demand series from Q1 2010 to Q3 2025 in black, with the three model forecasts overlaid for the 8-quarter test period (Q4 2023 to Q3 2025). The most notable observation is the sharp black spike reaching approximately 2,083 tonnes in Q2 2025, which represents the actual gold demand. This value is nearly double the historical average of around 1,100 tonnes. None of the models were able to predict this unusual increase.

Individual Model Forecasts

A: The ARIMAX forecast generally stays between 900 and 1,200 tonnes and is slightly higher than the other models due to the influence of the external regressors. However, it remains far below the actual spike, contributing to its highest RMSE (538.078).

B-a: The Ensemble forecast remains stable at around 1,050-1,200 tonnes throughout the test period. By averaging several models, it follows the historical pattern more closely but still fails to capture the sudden surge in demand. Its RMSE is 446.418.

B-b: The Bagged forecast is very similar to the Ensemble forecast, with both lines appearing almost identical on the plot. It achieves the lowest RMSE (437.318), indicating slightly better accuracy.

Summary

Overall, all three models substantially underestimated the Q2 2025 demand spike. This suggests that the large forecast errors are mainly driven by this exceptional event rather than poor performance throughout the entire test period. The Bagged and Ensemble models appear more stable, while ARIMAX is more dependence on the constant regressor assumption.

2. Cross Validation (CV) (set h = 12)

```
arimax_cv_wrapper <- function(y, h) {
  n <- length(y)
  x_in <- xreg_full[1:n, , drop = FALSE]
  x_out <- if ((n + h) <= nrow(xreg_full)) xreg_full[(n+1):(n+h), , drop = FALSE] else
    matrix(rep(tail(x_in, 1), h), nrow = h, byrow = TRUE,
            dimnames = list(NULL, colnames(xreg_full)))
  forecast(auto.arima(y, xreg = x_in, stepwise = TRUE, approximation = TRUE),
           xreg = x_out, h = h)
}

ensemble_fcfun <- function(y, h) {
  avg <- (forecast(auto.arima(y, stepwise = TRUE, approximation = TRUE), h = h)$mean +
         forecast(ets(y), h = h)$mean +
         snaive(y, h = h)$mean +
         stlf(y, h = h)$mean) / 4
  structure(list(mean = avg, x = y, method = "Ensemble"), class = "forecast")
}

bagged_fcfun <- function(y, h) {
  set.seed(42)
  forecast(baggedModel(y, bootstrapped_series = bld.mbb.bootstrap(y, 20)), h = h)
}

set.seed(42)
cv_arimax <- tsCV(GoldDemand, arimax_cv_wrapper, h = 12, initial = 20)
cv_ensemble <- tsCV(GoldDemand, ensemble_fcfun, h = 12, initial = 20)
cv_bagged <- tsCV(GoldDemand, bagged_fcfun, h = 12, initial = 20)

cv_results <- rbind(
  `A: ARIMAX` = c(CV_RMSE = sqrt(mean(cv_arimax^2, na.rm = TRUE)),
                  CV_MAE = mean(abs(cv_arimax), na.rm = TRUE)),
  `B-a: Ensemble` = c(CV_RMSE = sqrt(mean(cv_ensemble^2, na.rm = TRUE)),
                     CV_MAE = mean(abs(cv_ensemble), na.rm = TRUE)),
  `B-b: Bagged` = c(CV_RMSE = sqrt(mean(cv_bagged^2, na.rm = TRUE)),
                   CV_MAE = mean(abs(cv_bagged), na.rm = TRUE))
)

cat("\n=== Section C: Cross-Validation Results (h = 12) ===\n")
print(round(cv_results, 3))
```

Three forecasting functions were created for use with `tsCV()`. The `arimax_cv_wrapper` fits an ARIMAX model on each rolling training set using aligned external regressors (Price, Inflation, Rate, USD) and produces forecasts using future predictor values; when these were unavailable near the end of the sample, the last observed values were carried forward. Here, `n <- length(y)` defines the training window size, `x_in` selects the corresponding regressors for alignment, and `x_out` extracts the next `h` regressor values needed for forecasting.

The `ensemble_fcfun` function averages forecasts from ARIMA, ETS, Seasonal Naïve, and STL models, while the `bagged` function generates 20 moving block bootstrap samples, fits ETS models to each, and averages the forecasts.

Cross-validation used `tsCV()` with `h = 12` and `initial = 20`, meaning models were first trained on 20 observations and then updated one observation at a time. Forecast errors for horizons

1 to 12 were collected, and RMSE and MAE were computed by averaging across all rolling windows.

```

=== Section C: Cross-Validation Results (h = 12) ===
              CV_RMSE  CV_MAE
A: ARIMAX      328.937  222.410
B-a: Ensemble  248.361  158.120
B-b: Bagged    242.286  156.485

```

Output 9. Cross-Validation Results - CV RMSE and CV MAE

The cross-validation results support the findings from the holdout test set. Once again, the Bagged ETS model achieved the best forecasting accuracy, with the lowest CV RMSE (242.286) and CV MAE (156.485). The Ensemble model was a close second, producing a CV RMSE of 248.361 and a CV MAE of 158.120. The ARIMAX model had the highest errors, with a CV RMSE of 328.937 and a CV MAE of 222.410, showing weaker performance across different training periods.

The RMSE-by-horizon plot shows that ARIMAX errors rise quickly after horizon 5, exceeding 500 tonnes by horizon 12, meaning its accuracy drops as the forecast moves further into the future, likely due to increasing uncertainty in the predictors. In contrast, the Ensemble and Bagged models increase more slowly and stay close to each other, showing more stable performance across all horizons. Overall, both the holdout and cross-validation results suggest the Bagged ETS model (B = 20) performs best, with the Ensemble model close behind. While ARIMAX is useful for interpretation because it includes economic variables, it performs worst in accuracy, showing a trade-off between interpretability and predictive power.

```

data.frame(
  Horizon = 1:12,
  ARIMAX = cv_rmse_by_h(cv_arimax),
  Ensemble = cv_rmse_by_h(cv_ensemble),
  Bagged = cv_rmse_by_h(cv_bagged)
) %>%
  pivot_longer(~Horizon, names_to = "Model", values_to = "CV_RMSE") %>%
  ggplot(aes(x = Horizon, y = CV_RMSE, colour = Model, group = Model)) +
  geom_line(lwd = 0.5) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 1:12) +
  ggtitle("Cross-Validation RMSE by Forecast Horizon (h = 1 to 12)") +
  xlab("Forecast Horizon (Quarters)") +
  ylab("CV RMSE (Tonnes)") +
  theme_minimal() +
  guides(colour = guide_legend(title = "Model"))

```

A data frame is created with 12 forecast horizons (1–12), and `cv_rmse_by_h()` computes RMSE at each horizon across rolling windows to assess how accuracy changes over time. The data is reshaped using `pivot_longer()` for plotting, and `ggplot()` visualises RMSE against

horizon for each model, with lines and points showing performance trends across all 12 steps.

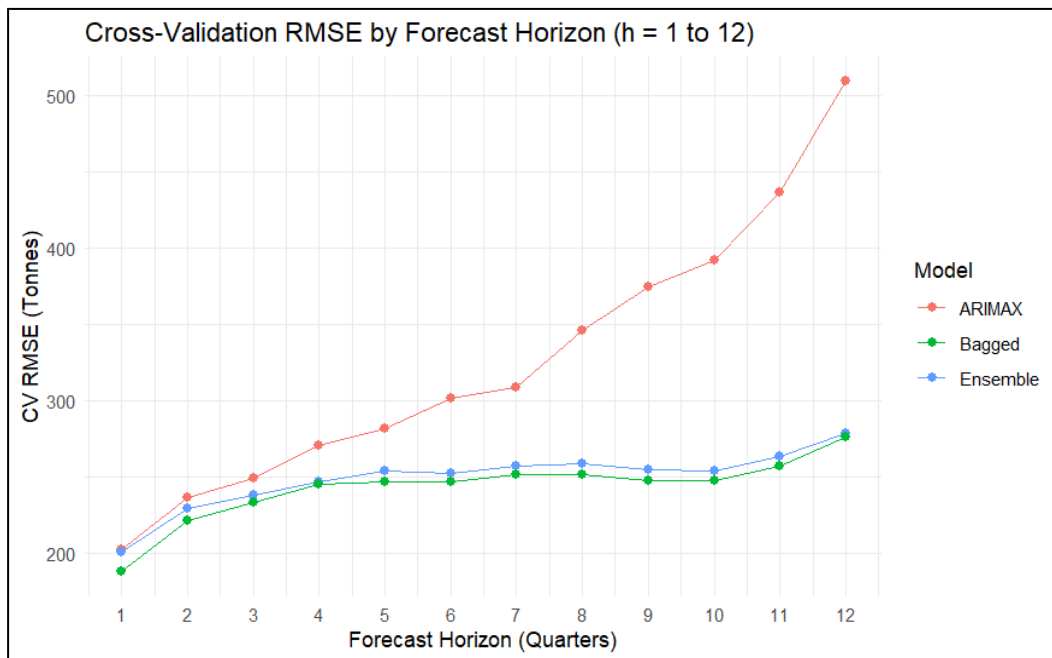


Figure 7. Cross validation RMSE by forecast ($h = 1$ to $h = 12$)

At $h = 1$, all three models perform similarly, with CV RMSE around 190–210 tonnes, showing strong short-term accuracy.

From $h = 2$ to 4, the models begin to separate slightly. ARIMAX increases from about 230 to 270 tonnes, while Bagged and Ensemble stay lower and closely grouped at 220–250 tonnes, still showing similar short-run performance.

From $h = 5$ to 8, the difference becomes clear. ARIMAX rises sharply from around 310 to 400 tonnes, while Bagged and Ensemble remain much more stable, increasing only slightly to about 260 tonnes. This suggests ARIMAX becomes less reliable as forecast uncertainty increases and regressors become less accurate.

From $h = 9$ to 12, ARIMAX deteriorates further, reaching about 510 tonnes at $h = 12$. In contrast, Bagged and Ensemble stay stable, ending around 270–275 tonnes, showing much slower error growth.

Across all horizons, Bagged is slightly better than Ensemble by a small margin (around 5–10 tonnes), suggesting a small but consistent advantage from bagging.

Summary

Overall, all models work similarly for short-term forecasts, but beyond $h = 5$, Bagged and Ensemble clearly outperform ARIMAX, which becomes increasingly unstable at longer horizons.

D Interpretation and Reflection

In Section A, we built an ARIMAX model, which is a *causal model*. It focused on understanding why gold demand changes, explicitly incorporating gold price, inflation, interest rates, and the USD index as predictor. Section B, in contrast, relied purely on historical patterns in the GoldDemand series itself through Ensemble and Bagged ETS methods, making them purely predictive models focused on *what* demand will be, without explaining the underlying drivers.

Strengths and Weaknesses of the ARIMAX Model (Causal Approach)

The key strength of the ARIMAX model is interpretability. The coefficient explains how each factor affects gold demand. For example, the results suggest that a 1% increase in interest rates was associated with an increase of about 76.52 tonnes in gold demand ($p = 0.015$), while a rise in inflation reduced demand by about 16.09 tonnes ($p = 0.024$).

These findings provide actionable insights for policymakers and economists who need to understand which macroeconomic forces drive gold demand, not just predict its level. A central bank deciding on gold reserve allocations, or a commodity trader assessing how a Fed rate decision might affect demand, would benefit directly from this type of causal model.

However, the main limitation is that it is hard to forecast into the future accurately. This is because ARIMAX needs future values of all the predictors (like inflation and interest rates). In this case, the last observed values were held constant, which is unrealistic, especially during periods like 2024-2025 when the economy surged a lot and gold demand spiked.

As a result, the ARIMAX model performed poorly in prediction, with the highest error rates (RMSE = 538.078 test-set, CV RMSE = 328.937). This shows that even though the model explains relationships well, it does not necessarily predict well.

Strengths and Weaknesses of Ensemble and Bagged ETS (Predictive Approach)

The Ensemble and Bagged models offer simplicity and stability as they require no external predictor data and can be deployed immediately using only the historical demand series. By averaging across multiple models or bootstrap replicates, they reduce forecast variance and produce more stable estimates. This is shown in the results, where the Bagged model performed best overall, achieving the lowest error (test RMSE = 437.318, CV RMSE = 242.286). These models also performed more consistently across different forecast horizons compared to ARIMAX which deteriorated sharply beyond horizon 5.

However, the disadvantage is that they cannot explain why changes happen. They only follow patterns in the historical data. So while they may predict demand well, they cannot tell us whether changes are caused by inflation, interest rates, or other economic forces.

Another limitation is that they can fail when the future is very different from the past. For example, all models missed the Q2 2025 spike (2,083 tonnes) because they were trained on more stable historical patterns and could not anticipate such a sudden structural change or policy changes.

Examples of Data Better Suited

Causal Models (ARIMAX)

Causal models are preferred when understanding the relationship matters as much as the forecast

1. Energy consumption forecasting using GDP, population, and temperature where governments need to understand which policies, such as carbon taxes or energy efficiency measures, can reduce energy use.
2. House price modelling includes interest rates, household income, and housing supply because policymakers need to see how changes in interest rates affect housing markets
3. Crop yield forecasting uses factors like rainfall, temperature, and fertiliser input where agricultural ministries intervene through specific conditions directly affect crop production

Purely Predictive Models (ETS/Bagging/Ensemble)

Purely predictive models are preferred when accuracy matter more

1. Retail demand forecasting at Store-Keeping Unit level involving thousands of products with no identifiable causes for demand changes, where pattern-based models scale efficiently
2. Short-term electricity load forecasting driven by time-of-day and seasonal patterns dominate and speed of prediction matters more than explanation
3. Web traffic and click-through rate forecasting is based on complex and often unknown processes, but historical patterns are stable enough to extrapolate

so the focus is on producing fast and accurate predictions rather than explaining why changes happen.

Conclusion

In summary, causal models like ARIMAX are most valuable when understanding and policy inference are the primary goal, as the coefficient estimates reveal the direction and magnitude of each economic driver's effect on gold demand. Purely predictive models like Ensemble and Bagged ETS are most valuable when forecast accuracy and operational deployment are the priority, as they consistently outperformed ARIMAX in both test-set and cross-validation evaluations. The two approaches are not mutually exclusive, a forecaster might use ARIMAX to communicate the economic story behind gold demand trends to stakeholders, while simultaneously using a Bagged ETS for day-to-day operational forecasting where accuracy matters most.

References

International Monetary Fund. (2023, January 31). *World Economic Outlook Update: Inflation peaking amid low growth*.

<https://www.imf.org/en/publications/weo/issues/2023/01/31/world-economic-outlook-update-january-2023>

World Gold Council. (2023). *Gold demand trends full year 2022*.

<https://www.gold.org/goldhub/research/gold-demand-trends/gold-demand-trends-full-year-2022>

World Gold Council. (2024). *Gold demand trends full year 2023*.

<https://www.gold.org/goldhub/research/gold-demand-trends/gold-demand-trends-full-year-2023>

International Monetary Fund. (2023). *World Economic Outlook update: Inflation and monetary tightening*. <https://www.imf.org/en/Publications/WEO>